

فاز ۱۴-۲: Hibernate و JPA – کامل و عمیق

Hibernate محبوب‌ترین فریمورک ORM توی جاواست. اگه Lazy Loading، Persistence Context و Cascade رو عمیق نفهمی، توی Spring Data JPA به مشکل می‌خوری.

۲.۱-۱۴ Hibernate چیه؟

Hibernate یه فریمورک (Object-Relational Mapping) ORM هست که وظیفه‌ش نقشه‌برداری خودکار از آبجکت‌های جاوا به جدول‌های دیتابیس.

چرا Hibernate به جای JDBC؟

ویژگی	JDBC	Hibernate
نوشتن SQL	دستی	خودکار
Mapping	دستی (ResultSet → Object)	خودکار
Cache	✗	Level 1 / Level 2 ✓
استقلال از دیتابیس	✗ وابسته به SQL	✓ با Dialect عوض میشه
مدیریت Transaction	دستی	خودکار

1 java

2 // روش JDBC

3 String sql = "INSERT INTO users (name, email) VALUES (?, ?)";

4 PreparedStatement ps = conn.prepareStatement(sql);

5 ps.setString(1, user.getName());

6 ps.setString(2, user.getEmail());

7 ps.executeUpdate();

8

9 // روش Hibernate

10 session.persist(user); // SQL تولید میشه خودکار

JPA vs Hibernate ۱۴-۲.۲

! سوال مصاحبه‌ای ۱: فرق JPA و Hibernate چیه؟

جواب:

ویژگی	JPA	Hibernate
ماهیت	استاندارد (Specification)	پیاده‌سازی (Implementation)
پکیج	javax.persistence / jakarta.persistence	org.hibernate
اینترفیس اصلی	EntityManager	Session
زبان Query	JPQL	HQL
رابطه	استاندارد رو تعریف می‌کنه	استاندارد JPA رو پیاده می‌کنه

نکته کلیدی: JPA اینترفیس، Hibernate پیاده‌سازی. می‌تونن پیاده‌سازی رو عوض کنی (مثل EclipseLink بدون تغییر کد JPA).

1 java

2 // روش JPA (استاندارد)

3 @PersistenceContext

4 private EntityManager entityManager;

5 entityManager.persist(user);

6

7 // روش Hibernate خالص

8 @Autowired

9 private SessionFactory sessionFactory;

10 sessionFactory.getCurrentSession().persist(user);

۱۴-۲.۳ کامپوننت‌های اصلی Hibernate

SessionFactory vs Session

ویژگی	SessionFactory	Session
دامنه	سطح اپلیکیشن (Singleton)	سطح درخواست
Thread-Safe	بله ✓	خیر ✗
هزینه ساخت	بالا	پایین
کاربرد	تولید Session	اجرای CRUD

1 java

2 // SessionFactory: یکی برای کل اپ

3 SessionFactory factory = new Configuration()

4 .configure("hibernate.cfg.xml")

5 .buildSessionFactory();

6

7 // Session: برای هر عملیات

8 Session session = factory.openSession();

9 Transaction tx = session.beginTransaction();

10

11 User user = new User("Ali", "ali@test.com");

12 session.persist(user);

13

14 tx.commit();

15 session.close();

Entity Persistence Context و چرخه حیات Entity ۱۴-۲.۴

چرخه حیات Entity

1		
2		
3	1. Transient (گذرا)	
4	↓ persist()	
5	2. Persistent (مدیریت شده)	
6	↓ detach() / close() / clear()	
7	3. Detached (جدا شده)	
8	↓ merge()	
9	2. Persistent (دوباره)	
10	↓ remove()	
11	4. Removed (حذف شده)	
12		
13		

```
1 java
```

```
2 //۱. Transient - Hibernate، شیء جدید - نمی‌شناسه
```

```
3 User user = new User("Ali", "ali@test.com");
```

```
4
```

```
5 //۲. Persistent - Hibernate می‌کنه مدیریتش
```

```
6 session.persist(user); // هست Persistence Context حالا توی
```

```
7
```

```
8 // Dirty Checking - تغییر خودکار
```

```
9 user.setName("Ali Reza"); // می‌زنه SQL خودکار commit نیست! موقع update() نیازی به
```

```
10
```

```
11 //۳. Detached - از Session شد خارج
```

```
12 session.close(); // دیگه مدیریت نمیشه user حالا
```

```
13
```

```
14 //۴. Removed
```

```
15 session.remove(user);
```

! سوال مصاحبه‌ای ۲: Persistence Context چیه؟

جواب: Persistence Context به حافظه موقت (First-Level Cache) هست که Hibernate توش Entityهای مدیریت شده رو نگه می‌داره. مزایاش:

1. Dirty Checking: تغییرات Entityها رو خودکار تشخیص میده.

2. First-Level Cache: اگه به Entity رو دوبار بخونی، فقط به SQL می‌زنه.

3. Write-Behind: SQLها رو تا زمان commit عقب می‌ندازه (Batching).

```

1 java
2 User user1 = session.get(User.class, 1L); // SQL می‌زنه
3 User user2 = session.get(User.class, 1L); // SQL نمی‌زنه! از Cache می‌خونه
4 System.out.println(user1 == user2); // true

```

۱۴-۲.۵ Inheritance (وراثت) توی Hibernate

Hibernate چهار استراتژی برای Map کردن وراثت داره:

۱. SINGLE_TABLE (پیش‌فرض)

همه کلاس‌ها توی یه جدول ذخیره میشن. یه ستون DTYPE نوع رو مشخص می‌کنه.

```

1 java
2 @Entity
3 @Inheritance(strategy = InheritanceType.SINGLE_TABLE)
4 @DiscriminatorColumn(name = "type", discriminatorType = DiscriminatorType.STRING)
5 public abstract class Payment {
6     @Id @GeneratedValue
7     private Long id;
8     private double amount;
9 }
10
11 @Entity
12 @DiscriminatorValue("CARD")
13 public class CardPayment extends Payment {
14     private String cardNumber;
15 }
16
17 @Entity
18 @DiscriminatorValue("CASH")
19 public class CashPayment extends Payment {
20     private String currency;
21 }

```

مزایا: سریع‌ترین (بدون JOIN)، ساده.

معایب: ستون‌های NULL زیاد، نمی‌تونی NOT NULL بذاری.

۲. JOINED

هر کلاس جدول خودش رو داره. با JOIN به هم وصل میشن.

```
1 java
2 @Entity
3 @Inheritance(strategy = InheritanceType.JOINED)
4 public abstract class Payment {
5     @Id @GeneratedValue
6     private Long id;
7     private double amount;
8 }
9
10 @Entity
11 public class CardPayment extends Payment {
12     private String cardNumber;
13 }
```

مزایا: نرمال‌سازی، NOT NULL راحت.

معایب: JOIN زیاد، کندتر.

۳. TABLE_PER_CLASS

هر کلاس جدول کامل خودش رو داره (حتی فیلدهای پدر).

```
1 java
2 @Entity
3 @Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
4 public abstract class Payment {
5     @Id @GeneratedValue
6     private Long id;
7     private double amount;
8 }
```

```

9
10 @Entity
11 public class CardPayment extends Payment {
12     private String cardNumber;
13     // id و amount هم توی این جدول تکرار میشن
14 }

```

مزایا: بدون JOIN برای هر کلاس.

معایب: UNION برای Queryهای پدر، تکرار ستون.

۴. MappedSuperclass

کلاس پدر جدول نداره، فقط فیلدها رو به ارث میده.

```

1 java
2 @MappedSuperclass
3 public abstract class BaseEntity {
4     @Id @GeneratedValue
5     private Long id;
6
7     @Column(name = "created_at")
8     private LocalDateTime createdAt;
9 }
10
11 @Entity
12 public class User extends BaseEntity {
13     private String name;
14 }

```


! سوال مصاحبه‌ای ۳: کدوم استراتژی وراثت بهتره؟

جواب:

استراتژی	سرعت	نرمال سازی	کاربرد
SINGLE_TABLE	سریع‌ترین ⚡	✗	وراثت کم، فیلدهای مشترک زیاد
JOINED	کند	✓	وراثت پیچیده، داده‌های زیاد
TABLE_PER_CLASS	متوسط	✗	هر کلاس مستقل
MappedSuperclass	-	-	فقط اشتراک فیلد (بدون Polymorphism)

توصیه: اگر Query پدر زیاد داری و تعداد کلاس‌ها کمه، SINGLE_TABLE. اگر نرمال سازی مهمه، JOINED.

۱۴-۲.۶ Association (انجمن) توی Hibernate

انواع رابطه

رابطه	Annotation	مثال
One-to-One	OneToOne@	User ↔ Profile
One-to-Many	OneToMany@	User → Orders
Many-to-One	ManyToOne@	Orders → User
Many-to-Many	ManyToMany@	Student ↔ Course

One-to-One .۱

```

1 java
2 @Entity
3 public class User {
4     @Id @GeneratedValue
5     private Long id;
6     private String name;
7
8     @OneToOne(cascade = CascadeType.ALL)
9     @JoinColumn(name = "profile_id", referencedColumnName = "id")
10    private Profile profile;
11 }
12
13 @Entity
14 public class Profile {
15     @Id @GeneratedValue
16     private Long id;
17     private String bio;
18
19     @OneToOne(mappedBy = "profile")
20     private User user;
21 }

```

One-to-Many / Many-to-One .۲

```

1 java
2 @Entity
3 public class User {
4     @Id @GeneratedValue
5     private Long id;
6     private String name;
7
8     @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
9     private List<Order> orders = new ArrayList<>();
10
11     // متد کمکی
12     public void addOrder(Order order) {

```

14-2-Hibernate-JPA

```
13     orders.add(order);
14     order.setUser(this);
15 }
16 }
17
18 @Entity
19 public class Order {
20     @Id @GeneratedValue
21     private Long id;
22     private String product;
23
24     @ManyToOne(fetch = FetchType.LAZY)
25     @JoinColumn(name = "user_id")
26     private User user;
27 }
```

Many-to-Many .۳

```
1 java
2 @Entity
3 public class Student {
4     @Id @GeneratedValue
5     private Long id;
6     private String name;
7
8     @ManyToMany(cascade = CascadeType.ALL)
9     @JoinTable(
10         name = "student_course",
11         joinColumns = @JoinColumn(name = "student_id"),
12         inverseJoinColumns = @JoinColumn(name = "course_id")
13     )
14     private List<Course> courses = new ArrayList<>();
15 }
16
17 @Entity
18 public class Course {
19     @Id @GeneratedValue
20     private Long id;
```

```

21 private String title;
22
23 @ManyToMany(mappedBy = "courses")
24 private List<Student> students = new ArrayList<>();
25 }

```

! سوال مصاحبه‌ای ۴: mappedBy چیه و چرا لازمه؟

جواب: mappedBy به Hibernate می‌گه که این سمت رابطه، مالک نیست و از سمت دیگه پیروی می‌کنه. بدون mappedBy، Hibernate دو جدول واسط می‌سازه و دوبار SQL می‌زنه!

```

1 java
2 // غلط - دو جدول واسط ساخته میشه ✗
3 @OneToMany
4 private List<Order> orders;
5
6 // درست - یه جدول واسط، مالکیت با ✓
7 @OneToMany(mappedBy = "user")
8 private List<Order> orders;

```

قانون: سمت @ManyToOne همیشه مالک. سمت @OneToMany باید mappedBy داشته باشه.

۱۴-۲.۷ FetchType (نوع واکشی)

! سوال مصاحبه‌ای ۵: فرق EAGER و LAZY چیه؟

جواب:

ویژگی	LAZY	EAGER
زمان واکشی	موقع دسترسی	همون لحظه لود Entity
کارایی	✓ بهتر	✗ کندتر
خطر	LazyInitializationException	N+1 Problem
پیش فرض	OneToMany, @ManyToMany@	ManyToOne, @OneToOne@

1 java

```

2 // LAZY - میزنه SQL، صدا بزنی getOrders() فقط وقتی
3 @OneToMany(fetch = FetchType.LAZY)
4 private List<Order> orders;
5
6 // EAGER - ها هم لود میشن: User, Order همون لحظه لود
7 @ManyToOne(fetch = FetchType.EAGER)
8 private User user;
```

مشکل LazyInitializationException

1 java

```

2 // ❌ خطا
3 User user = session.get(User.class, 1L);
4 session.close(); // Session بسته شد
5 user.getOrders().size(); // LazyInitializationException!
6
7 // ✅ دسترسی بزنی Session راه حل ۱: توی
8 User user = session.get(User.class, 1L);
9 user.getOrders().size(); // Session هنوز بازه
10 session.close();
11
12 // ✅ JOIN FETCH: راه حل ۲
13 @Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
14 User findWithOrders(@Param("id") Long id);
```

مشکل N+1

1 java

```

2 // ❌ N+1 Problem
3 List<User> users = userRepository.findAll(); // ۱ Query
4 for (User u : users) {
5     u.getOrders().size(); // N Query اضافه!
6 }
7 // مجموع: N+1 Query
8
```

```

9 // ✓ راه حل: JOIN FETCH
10 @Query("SELECT DISTINCT u FROM User u JOIN FETCH u.orders")
11 List<User> findAllWithOrders(); // ۱ فقط Query

```

! سوال مصاحبه‌ای ۶: N+1 Problem چیست و چگونه حل می‌کنیم؟

جواب: N+1 یعنی برای لود کردن N تا Entity، به جای ۱ N+1، Query تا Query زده می‌شود.

راه حل‌ها:

1. JOIN FETCH توی JPQL

2. @EntityGraph توی Spring Data JPA

3. @BatchSize برای لود دسته‌ای

4. @Fetch(FetchMode.SUBSELECT)

5. تنظیم hibernate.default_batch_fetch_size

```

1 java
2 // ۱ راه حل: JOIN FETCH
3 @Query("SELECT u FROM User u JOIN FETCH u.orders")
4 List<User> findAllWithOrders();
5
6 // ۲ راه حل: EntityGraph
7 @EntityGraph(attributePaths = {"orders"})
8 List<User> findAll();
9
10 // ۳ راه حل: BatchSize
11 @OneToMany
12 @BatchSize(size = 10)
13 private List<Order> orders;

```

۲.۸-۱۴ Cascade (آبشاری)

! سوال مصاحبه‌ای ۷: Cascade چیه و انواعش؟

جواب: Cascade یعنی عملیات روی Entity پدر، به فرزندان هم اعمال بشه.

نوع Cascade	توضیح
PERSIST	ذخیره پدر → فرزندان هم ذخیره
MERGE	آپدیت پدر → فرزندان هم آپدیت
REMOVE	حذف پدر → فرزندان هم حذف
REFRESH	رفرش پدر → فرزندان هم رفرش
DETACH	جدا کردن پدر → فرزندان هم جدا
ALL	همه موارد بالا

1 java

2 //  با Cascade

3 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)

4 private List<Order> orders = new ArrayList<>();

5

6 // استفاده

7 User user = new User("Ali");

8 Order order1 = new Order("Book");

9 Order order2 = new Order("Pen");

10 user.addOrder(order1);

11 user.addOrder(order2);

12

13 session.persist(user); // Order ها هم خودکار ذخیره میشن

orphanRemoval

```

1 java
2 @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
3 private List<Order> orders = new ArrayList<>();
4
5 حذف از لیست = حذف از دیتابیس //
6 user.getOrders().remove(0); // Order همیشه حذف خودکار

```

! سوال مصاحبه‌ای ۸: فرق Cascade REMOVE و orphanRemoval چیه؟

جواب:

ویژگی	Cascade REMOVE	orphanRemoval
حذف پدر	فرزندان حذف میشن	فرزندان حذف میشن
حذف از لیست	✗ حذف نمیشن	✓ حذف میشن
جدا کردن رابطه	✗	✓ (اگه setUser(null) کنی)

```

1 java
2 // Cascade.REMOVE
3 user.getOrders().remove(order); // Order توی DB می‌مونه
4 session.remove(user); // Order میشن حالا
5
6 // orphanRemoval
7 user.getOrders().remove(order); // Order از DB همیشه حذف خودکار

```


۱۴-۲.۹ Hibernate Caching توی

First-Level Cache (الزامی)

همون Persistence Context توی Session نگه داشته میشه.

```
1 java
2 User u1 = session.get(User.class, 1L); // SQL
3 User u2 = session.get(User.class, 1L); // Cache
```

Second-Level Cache (اختیاری)

توی SessionFactory نگه داشته میشه. بین Session ها مشترکه.

```
1 xml
2 <!-- hibernate.cfg.xml -->
3 <property name="hibernate.cache.use_second_level_cache">true</property>
4 <property name="hibernate.cache.region.factory_class">
5   org.hibernate.cache.ehcache.EhCacheRegionFactory
6 </property>
```

```
1 java
2 @Entity
3 @Cacheable
4 @org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
5 public class User { ... }
```

Query Cache

```
1 java
2 Query query = session.createQuery("FROM User WHERE name = :name");
3 query.setParameter("name", "Ali");
4 query.setCacheable(true); // فعال کردن Query Cache
```

۱۰-۲-۱۴ Locking (قفل‌گذاری)

Optimistic Locking

با به ستون version کار می‌کنه. تداخل رو موقع Commit تشخیص میده.

```

1 java
2 @Entity
3 public class Product {
4     @Id @GeneratedValue
5     private Long id;
6
7     @Version
8     private int version;
9
10    private int quantity;
11 }

1 java
2 // Thread 1
3 Product p1 = session.get(Product.class, 1L);
4 p1.setQuantity(5);
5 // Thread 2 همزمان
6 Product p2 = session.get(Product.class, 1L);
7 p2.setQuantity(10);
8
9 // Thread 1 commit → version = 2
10 // Thread 2 commit → OptimisticLockException!

```

Pessimistic Locking

دیتابیس رو قفل می‌کنه.

```
1 java
2 // Lock on Read
3 Product p = session.get(Product.class, 1L, LockMode.PESSIMISTIC_WRITE);
4
5 // Lock on Query
6 Query query = session.createQuery("FROM Product WHERE id = :id");
7 query.setLockMode("p", LockMode.PESSIMISTIC_WRITE);
```

! سوال مصاحبه‌ای ۹: فرق Optimistic Locking و Pessimistic Locking چیه؟

جواب:

ویژگی	Optimistic	Pessimistic
روش	version column	DB Lock
کارایی	بالا ✓	پایین ✗
تداخل	موقع Commit	موقع Read
کاربرد	Read زیاد، Write کم	Write زیاد، تداخل زیاد
خطا	OptimisticLockException	زمان انتظار

Flush و Transaction ۱۴-۲.۱۱

Flush چیه؟

Flush یعنی SQL های Hibernate معلق رو به دیتابیس بفرسته (ولی Commit نمیشه).

```
1 java
2 session.persist(user); // زده SQL هنوز
3 session.flush(); // (نه Commit ولی) زده شد INSERT حالا
4 tx.commit(); // Commit
```

FlushMode

توضیح	حالت
قبل از Query و Commit	AUTO (پیش فرض)
فقط موقع Commit	COMMIT
دستی با flush()	MANUAL
قبل از هر Query	ALWAYS

! سوال مصاحبه‌ای ۱۰: فرق persist, save, saveOrUpdate, merge چیه؟

جواب:

توضیح	Detached	Transient	برگردوند	متد
JPA استاندارد	خطا ✗	✓ ذخیره	void	()persist
Hibernate	✗ INSERT جدید	✓ ذخیره	ID	()save
Hibernate	✓ آپدیت	✓ ذخیره	void	()saveOrUpdate
JPA استاندارد	✓ آپدیت	✓ ذخیره	Entity	()merge
Hibernate	✓ آپدیت	✗ خطا	void	()update

1 java

2 // *persist* فقط - *Transient*

3 User user = new User("Ali");

4 session.persist(user); // *INSERT*

5

6 // *merge* - هر دو حالت

7 User detached = ...; // از *Session* قبلی

8 User managed = session.merge(detached); // *SELECT + UPDATE*

JPQL vs HQL vs Native SQL ۱۴-۲.۱۲

```

1 java
2 // JPQL (استاندارد)
3 @Query("SELECT u FROM User u WHERE u.age > :age")
4 List<User> findOlderThan(@Param("age") int age);
5
6 // HQL (Hibernate-specific)
7 Query query = session.createQuery("FROM User u WHERE u.name LIKE :name");
8 query.setParameter("name", "%Ali%");
9
10 // Native SQL
11 @Query(value = "SELECT * FROM users WHERE age > ?1", nativeQuery = true)
12 List<User> findOlderThanNative(int age);

```

! سوال مصاحبه‌ای ۱۱: JPQL و SQL چه فرقی دارند؟

جواب:

ویژگی	JPQL	SQL
موجودیت‌ها	Field و Entity	Column و Table
حساسیت	به Case حساس	معمولاً نه
Polymorphism	✓	✗
قابل حمل	✓ بین DBها	✗

```

1 java
2 // JPQL
3 "SELECT u FROM User u WHERE u.name = 'Ali'"
4
5 // SQL معادل
6 "SELECT * FROM users WHERE name = 'Ali'"

```

Order و User : نمونه کامل: ۱۴-۲.۱۳

```

1 java
2 @Entity
3 @Table(name = "users")
4 public class User {
5
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @Column(nullable = false, length = 100)
11    private String name;
12
13    @Column(unique = true)
14    private String email;
15
16    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL,
17        orphanRemoval = true, fetch = FetchType.LAZY)
18    private List<Order> orders = new ArrayList<>();
19
20    @OneToOne(cascade = CascadeType.ALL)
21    @JoinColumn(name = "profile_id")
22    private Profile profile;
23
24    @ManyToMany(cascade = CascadeType.PERSIST)
25    @JoinTable(name = "user_role",
26        joinColumns = @JoinColumn(name = "user_id"),
27        inverseJoinColumns = @JoinColumn(name = "role_id"))
28    private Set<Role> roles = new HashSet<>();
29
30    @Version
31    private int version;
32
33    @CreationTimestamp
34    private LocalDateTime createdAt;
35
36    @UpdateTimestamp

```

14-2-Hibernate-JPA

```
37 private LocalDateTime updatedAt;
38
39 // متدهای کمکی
40 public void addOrder(Order order) {
41     orders.add(order);
42     order.setUser(this);
43 }
44
45 public void removeOrder(Order order) {
46     orders.remove(order);
47     order.setUser(null);
48 }
49
50 // Getters/Setters
51 }
52
53 @Entity
54 @Table(name = "orders")
55 public class Order {
56
57     @Id
58     @GeneratedValue(strategy = GenerationType.IDENTITY)
59     private Long id;
60
61     private String product;
62     private double amount;
63
64     @ManyToOne(fetch = FetchType.LAZY)
65     @JoinColumn(name = "user_id", nullable = false)
66     private User user;
67
68     // Getters/Setters
69 }
```

۱۴-۲-۱۴ جمع‌بندی – نکات طلایی مصاحبه

1. JPA استاندارد، Hibernate پیاده‌سازی.

2. Persistence Context = First-Level Cache – توی Session زندگی می‌کنه.

3. mappedBy سمت مالکیت رو مشخص می‌کنه – بدونش دو جدول واسط ساخته میشه.

4. LAZY پیش‌فرض، EAGER@، OneToMany، پیش‌فرض @ManyToOne.

5. N+1 Problem با JOIN FETCH یا EntityGraph حل میشه.

6. Cascade عملیات پدر → فرزندان، orphanRemoval حذف از لیست → حذف از DB.

7. SINGLE_TABLE سریع‌ترین استراتژی وراثت.

8. Optimistic Locking با @Pessimistic، Version با LockMode.

9. persist برای merge، Transient برای Detached.

10. Flush SQL – Commit ≠ Flush می‌زنه، Commit تأیید می‌کنه.

نکته طلایی: توی مصاحبه اگه پرسیدن "چرا Hibernate استفاده می‌کنی؟" بگو: "چون ORM باعث میشه کد دیتابیس ننویسم، Dirty Checking خودکار دارم، و با Dialect می‌تونم دیتابیس رو عوض کنم بدون تغییر کد. مهم‌تر از همه، Persistence Context و Caching کارایی رو بالا می‌بره."